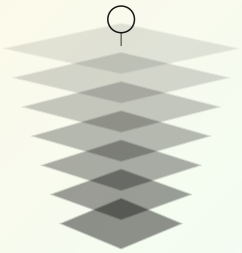
UNIT 1 · FOUNDATIONS AND PROCESS

CSC 4305 · Session 02 of 30

Plan-Driven Process Models



Waterfall, the V-model, incremental delivery, and the spiral model — what each one assumes about the world, and how to recognise the conditions under which each is the correct professional choice.

DATE	MEETING TIME	DURATION
Friday, 11 September 2026	10:00 – 11:20 AM	80 minutes

5 learning outcomes · **80** minutes · **6** timetabled activities · **5** figures and tables

1 Learning Outcomes

By the end of this session, a student who has done the assigned preparation will be able to:

LO 2.1
Draw the waterfall model from memory, labelling all phases and the feedback paths that the original 1970 paper actually included.

LO 2.2
Explain the V-model as a mapping between specification levels and test levels, and identify which test level validates which artifact.

LO 2.3
Contrast incremental delivery with incremental development, and state the customer-facing consequence of each.

LO 2.4
Describe the spiral model's four quadrants and explain how risk drives iteration length.

LO 2.5
Given a project description, select a process model and defend the choice against two named alternatives using at least three project characteristics.

2 Session Plan

0:00 – 0:08

Homework check-in; the four attributes of good software from the reading

DISCUSSION

8 min

0:08 – 0:30

Lecture: waterfall — what Royce actually wrote, and what industry did with it

INSTRUCTOR

22 min

0:30 – 0:45

Lecture: the V-model and the traceability it enforces

INSTRUCTOR

15 min

0:45 – 1:00

Lecture: incremental and spiral models; risk-driven iteration

INSTRUCTOR

15 min

1:00 – 1:15

In-class exercise: process selection for four contrasting projects

GROUPS OF 4

15 min

1:15 – 1:20

Debrief and preview of agile methods

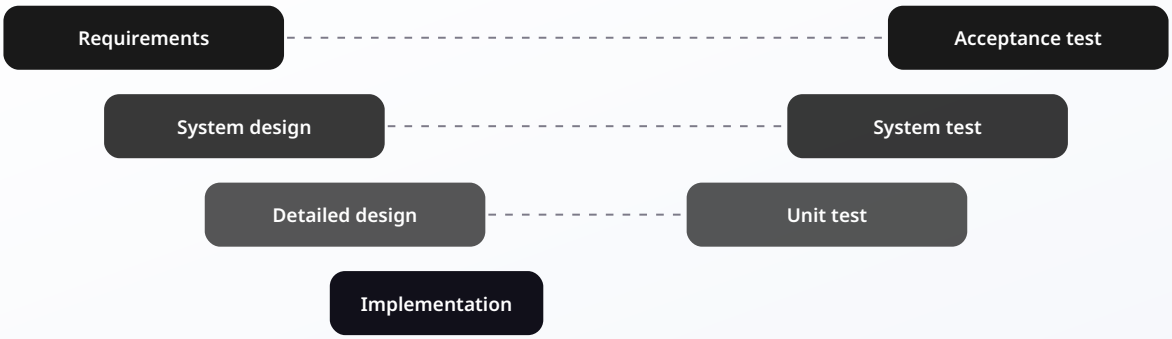
INSTRUCTOR

5 min

3

Lecture Notes

THE V-MODEL: EVERY SPECIFICATION DOCUMENT HAS A MATCHING TEST



The dashed lines are the model's whole argument — a test is planned when its specification is written, not after the code exists.

2.1 Waterfall: the model everyone criticises and almost nobody has read

Winston Royce's 1970 paper is routinely cited as the origin of the waterfall model. It is worth knowing that Royce presented the strictly sequential version as a starting point and then spent the rest of the paper explaining that it is **risky and invites failure**, before proposing feedback loops, prototyping, and customer involvement as remedies. The caricature that dominated industry for two decades is a misreading.

The sequential phases are: Requirements → Design → Implementation → Integration and testing → Deployment → Maintenance. Each phase is signed off before the next begins, and each produces a document that becomes the input to the next.

What waterfall assumes. (1) Requirements can be known and stabilised in advance. (2) The cost of change grows sharply with time, so front-loading analysis is economically rational. (3) The customer can evaluate a written specification. (4) The team is large enough or distributed enough that documented handoffs are cheaper than continuous conversation.

When those assumptions hold. Regulated and safety-critical development, where an auditor must see the specification that a test traces back to. Contract development where scope defines payment. Systems integration where you must agree interfaces with three other organisations twelve months before anyone writes code. Any of these makes waterfall a defensible engineering choice, not a historical relic.

When they fail. Whenever the customer will only recognise what they want by seeing it. This is the normal condition for interactive business and consumer software, which is why agile methods displaced waterfall in that segment.

2.2 The V-model: specification and test as mirror images

The V-model takes the waterfall sequence and bends it into a V. The descending arm is specification, moving from the general to the particular. The ascending arm is testing, moving from the particular back to the general. Each level on the left is validated by exactly one level on the right, and the pairing is the point of the model.

User requirements / concept of operations	System requirements specification
VERIFIED BY (RIGHT ARM) Acceptance testing	VERIFIED BY (RIGHT ARM) System testing
THE QUESTION IT ANSWERS Does this system solve the user's actual problem?	THE QUESTION IT ANSWERS Does the assembled system meet its written requirements?
Architectural design	**Detailed module design**
VERIFIED BY (RIGHT ARM) Integration testing	VERIFIED BY (RIGHT ARM) Unit testing
THE QUESTION IT ANSWERS Do the components work correctly together across their interfaces?	THE QUESTION IT ANSWERS Does each module do what its design says?
Code	
VERIFIED BY (RIGHT ARM) Static analysis, review	
THE QUESTION IT ANSWERS Is the implementation itself sound?	

→ **The practical payoff.** The V-model makes an uncomfortable question answerable: for any test you have written, which specification statement does it verify? And for any requirement, which test proves it? A requirement with no test is unverified. A test with no requirement is either dead work or an undocumented requirement. Both are defects in the process. This is called **bidirectional traceability**, and it is the reason the V-model dominates automotive, medical device, and avionics development.

2.3 Incremental development and incremental delivery

These two are constantly confused and the distinction matters to your customer.

Incremental development

the system is built in increments, but nothing is released until the whole is complete. The team gets feedback from its own integration; the customer does not.

Incremental delivery

each increment is deployed to real users and put into service. The customer gets value early, and the team gets feedback from actual use, which is the only feedback that reliably exposes wrong requirements.

Incremental delivery costs more per increment: each release needs deployment, documentation, training, and support. It buys you the ability to be wrong cheaply. Order the increments by risk and by value: build the increment that answers your scariest open question first, not the one that is easiest.

2.4 Boehm's spiral model: process driven by risk

Barry Boehm's 1988 spiral model is best understood not as a process but as a meta-process: it tells you how to choose what to do next. Each loop of the spiral passes through four quadrants.

1. **Determine objectives, alternatives, constraints** — what are we trying to achieve in this loop?
2. **Identify and resolve risks** — what could kill this project? Prototype, benchmark, or investigate until the risk is understood or retired.
3. **Develop and verify** — build the deliverable for this loop, using whatever local process fits (which may itself be waterfall).

4 **Plan the next iteration** — review with stakeholders and commit to the next loop, or stop.

The controlling insight is that **the largest unretired risk determines what you do next**. If you do not know whether the offline synchronisation will work, you do not spend six weeks perfecting the login screen. Every modern process, agile included, inherits this idea even when it does not draw the spiral.

2.5 A decision heuristic

PROJECT CHARACTERISTIC	PUSHES TOWARD
Requirements stable and externally fixed (regulation, contract, standard)	Waterfall / V-model
Safety-critical; certification evidence required	V-model
Requirements genuinely unknown until users touch the system	Incremental delivery / agile
Novel technology; major technical uncertainty	Spiral, prototype the risk first
Large distributed team, multiple organisations, fixed interfaces	Plan-driven with agile within teams
Small co-located team, accessible customer	Agile

Real organisations mix. A hospital information system may specify and certify its clinical decision module under a V-model while running its appointment-booking web front end on two-week agile sprints. Choosing one process for the whole project because it is the one you know is a failure of judgement, not a simplification.

4 Worked Practical Example

Scenario. The Ministry of Health commissions a national vaccination register. Two subsystems, one project.

Subsystem A — the immunisation record store and its clinical rules engine. Records are legally required to be retained for 30 years. Regulators must be able to audit which requirement each test verifies. The data model is defined by an existing WHO standard and will not change during development. Interfaces to two other government systems must be agreed a year in advance.

Subsystem B — the health worker tablet application used at village outreach sessions. Nobody, including the health workers themselves, can yet describe what the workflow should be. Connectivity is intermittent. The application must be usable by a nurse holding an infant in one arm.

Requirements volatility

SUBSYSTEM A

Low — fixed by standard and law

SUBSYSTEM B

Very high — discovered through use

Cost of a late defect

SUBSYSTEM A

Severe: legal and clinical

SUBSYSTEM B

Moderate: recoverable in the next release

Evidence required

SUBSYSTEM A

Full traceability for audit

SUBSYSTEM B

Usability evidence from field trials

Feedback available before release

SUBSYSTEM A

From reviews of documents

SUBSYSTEM B

Only from nurses using it in a village

Selected model

SUBSYSTEM A

V-model, with formal reviews at each level

SUBSYSTEM B

Incremental delivery, two-week increments, field trial each month

The integration problem. The two subsystems must meet. The resolution is to treat the interface between them as an artifact belonging to the plan-driven side: specify it early, version it, and let Subsystem B iterate freely *behind* a stable interface. This is the single most important structural move in mixed-process projects, and it is a design decision as much as a process decision. We will return to it in Session 12 when we discuss architecture.

→ **The defensible answer.** There is no single correct process for this project. There is a correct *justification*: the process choice must follow from requirements volatility, the cost of late defects, the evidence the project must produce, and the feedback actually available. An exam answer that names a model without those four elements is incomplete.

5 In-Class Exercise

Process selection (15 minutes, groups of four). Each group receives all four projects below and must assign a process model to each, then defend one assignment to the class.

1. **Project A.** Flight control software for an agricultural drone that sprays pesticide over cocoa plantations. A software fault could injure workers. Certification by the civil aviation authority is required before commercial use.
2. **Project B.** A mobile-money savings application for a fintech startup in Abidjan. The founders have three competing hypotheses about what customers actually want and eight months of funding.
3. **Project C.** Replacement of a 20-year-old COBOL payroll system for a company of 4,000 employees. Behaviour must match the old system exactly, including its documented quirks. Both systems will run in parallel for one full payroll cycle.

4 **Project D.** A machine-learning system that reads handwritten field notes from agricultural extension officers. Nobody yet knows whether the required accuracy is achievable at all.

For each project produce: the chosen model; three project characteristics that drove the choice; one named alternative and the specific reason you rejected it; and the largest risk your chosen process leaves unaddressed.

→ **Discussion points to expect.** Project D is the interesting one. Its dominant characteristic is feasibility uncertainty, not requirements uncertainty — so the first loop of a spiral should be a two-week feasibility spike on a labelled sample, with an explicit decision to stop if accuracy is unreachable. A team that proposes two-week sprints for Project D without a kill criterion has confused iteration with risk management. Project C is a trap in the other direction: it looks like a plan-driven project because the requirements exist, but they exist only as undocumented behaviour in 20-year-old code, so requirements *recovery* is the real work.

6 Homework / Deliverable

- Due before Session 4 (Friday 18 September), pushed to your personal repository as s02-process.md.

☐ **Diagram.** Draw the V-model for SANTE-BASSAM (the semester project). Label all five levels on each arm. For each level on the left arm, write one concrete example of an artifact you expect your team to produce this semester. Export as PNG or PDF and commit it.

☐ **Comparative writing (600–800 words).** Argue for the process model your team should use for SANTE-BASSAM. Your argument must address: requirements volatility, the availability of a real customer for feedback, the consequences of a defect in a health record, and the fact that your team has fourteen weeks and no budget. Explicitly reject one alternative model with a reason specific to this project — not a generic reason.

☐ **Reading.** Sommerville Chapter 2 (Software Processes), sections 2.1–2.3. Optional but recommended: Royce's original 1970 paper, *Managing the Development of Large Software Systems* — nine pages, freely available.

→ **Assessment link.** This deliverable becomes Section 2 of your team's Project Plan, due Session 5. Write it as if it will be read by a client, because a version of it will be.

7 Key Terms

waterfall model · V-model · bidirectional traceability · incremental development · incremental delivery · spiral model · risk exposure · prototype · requirements volatility · process tailoring

8 Readings and References

Sommerville, I. *Software Engineering*, 10th ed. — Chapter 2, sections 2.1–2.3.

Royce, W. Managing the Development of Large Software Systems. *Proc. IEEE WESCON*, 1970.

Boehm, B. A Spiral Model of Software Development and Enhancement. *IEEE Computer* 21(5), 1988.

Pressman and Maxim Chapter 2 (Process Models).